

|1 +-1 éé1 èè1 êê1 àà1 îî1 ôô1 ûû1 çç1 ÉÉ1



ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

Département Génie Informatique — 3^e Année

Introduction aux Réseaux — Semestre 2, 2025-2026

Business Core as a Service

BCAAS

*Une infrastructure générique de gestion des métiers
inspirée de l'ingénierie des protocoles réseaux*

Business Capabilities

Context & Policy

Tenant & Routing

Transport & Messaging

Infrastructure

Réalisé par :

ETCHOME Chris

[Coéquipier 2]

[Coéquipier 3]

Sous la supervision de l'équipe pédagogique du cours d'Introduction aux Réseaux

Résumé

Ce rapport présente la conception et le développement de BCAAS (*Business Core as a Service*), une plateforme générique multi-tenant dédiée à la gestion standardisée des processus métier. Le projet s'inscrit dans le cadre du cours d'Introduction aux Réseaux de la troisième année de Génie Informatique à l'École Nationale Supérieure Polytechnique de Yaoundé.

L'idée centrale est de modéliser un système d'information métier en s'appuyant rigoureusement sur les concepts de l'ingénierie des protocoles réseaux : pile de couches, encapsulation, routage, adressage, qualité de service et séparation entre plan de contrôle et plan de données. Cette analogie, formalisée mathématiquement, sert de méthode de conception et guide chaque décision architecturale.

La solution implémentée repose sur un **monolithe modulaire** Spring Boot 3.5 / Java 21, organisé selon l'architecture hexagonale et le *Domain-Driven Design*. Elle utilise PostgreSQL avec Liquibase, Spring Security et JWT pour l'authentification multi-tenant, Redis pour le cache, Apache Kafka pour les notifications asynchrones et SpringDoc OpenAPI pour la documentation automatique de l'API. Le frontend, une *Progressive Web App* développée avec Next.js 16 et React 19, combine un site vitrine, une documentation interactive, un assistant conversationnel et un tableau de bord d'administration.

Mots-clés : Business Core, multi-tenant, protocoles réseaux, modèle OSI, encapsulation, isolation, Spring Boot, API REST, JWT, Next.js.

Abstract

This report presents the design and development of BCAAS (*Business Core as a Service*), a generic multi-tenant platform dedicated to the standardized management of business processes. The project was carried out for the *Introduction to Networks* course of the third year of Computer Engineering at the National Advanced School of Engineering of Yaoundé.

The core idea is to model a business information system by rigorously relying on the concepts of network protocol engineering: layered stacks, encapsulation, routing, addressing, quality of service, and the separation between control plane and data plane. This analogy, given a formal mathematical treatment, is used as a design method and drives every architectural decision.

The implemented solution is a **modular monolith** built with Spring Boot 3.5 / Java 21, structured according to hexagonal architecture and Domain-Driven Design. It relies on PostgreSQL with Liquibase, Spring Security with JWT for multi-tenant authentication, Redis for caching, Apache Kafka for asynchronous notifications, and SpringDoc OpenAPI for automatic API documentation. The frontend, a Progressive Web App built with Next.js 16 and React 19, combines a landing site, interactive documentation, a conversational assistant, and an administration dashboard.

Keywords: Business Core, multi-tenancy, network protocols, OSI model, encapsulation, isolation, Spring Boot, REST API, JWT, Next.js.

Table des matières

Table des figures

Liste des tableaux

Chapitre 1

Introduction générale

1.1 Contexte du projet

Dans le domaine du génie logiciel, chaque nouveau projet métier — qu’il s’agisse d’une pharmacie, d’une agence immobilière, d’un cabinet médical, d’une auto-école ou d’une école — reconstruit invariablement les mêmes fondations : gestion des acteurs, des ressources, des règles, des workflows, des notifications et de la traçabilité. Cette duplication systématique représente un gaspillage considérable en temps de développement, en maintenance et en cohérence architecturale.

Parallèlement, l’ingénierie des réseaux a résolu depuis des décennies un problème structurellement similaire : faire communiquer de façon fiable, sécurisée et interopérable des entités hétérogènes sur une infrastructure partagée. Le modèle OSI (*Open Systems Interconnection*) et la suite TCP/IP proposent une décomposition en couches aux responsabilités clairement délimitées, un mécanisme d’encapsulation des données, un adressage précis des destinataires, une qualité de service différenciée et une séparation nette entre plan de contrôle et plan de données.

1.2 Problématique

Question directrice

Comment concevoir un noyau métier générique, réutilisable et configurable qui permette à n’importe quelle instance métier de fonctionner sans reconstruire les fondations, en s’appuyant formellement sur les concepts de l’ingénierie des protocoles réseaux ?

1.3 Objectifs

Les objectifs du projet sont les suivants :

1. Établir une analogie formelle et rigoureuse entre les concepts réseaux (couches OSI, encapsulation, routage, QoS, multi-tenant) et les composants d'une architecture métier générique.
2. Concevoir une architecture modulaire multi-tenant organisée en cinq couches protocolaires.
3. Implémenter une API REST de gestion d'instances métier reposant sur un modèle de données concret et réutilisable.
4. Documenter et exposer l'API via Swagger UI (SpringDoc OpenAPI).
5. Développer un frontend Next.js comprenant un site vitrine, une documentation interactive et un tableau de bord d'administration.

1.4 Structure du document

Ce rapport est organisé en treize chapitres. Le ?? dresse l'état de l'art des plateformes génériques et des architectures multi-tenant. Le ?? formalise mathématiquement l'analogie entre réseaux et métier. Le ?? présente l'architecture globale. Le ?? détaille la conception des composants et du modèle de données. Le ?? illustre le protocole d'échange sur un cas métier concret. Le ?? traite de la stratégie multi-tenant et de l'isolation. Le ?? décrit la sécurité. Le ?? aborde l'observabilité et la QoS métier. Le ?? décrit l'implémentation technique. Le ?? présente le frontend. Le ?? couvre les tests et la validation. Enfin, le ?? conclut et ouvre sur des perspectives.

Chapitre 2

État de l'art

2.1 Plateformes métier génériques

L'approche consistant à factoriser la logique commune de plusieurs domaines métier dans une plateforme partagée n'est pas nouvelle. On distingue plusieurs catégories :

ERP (Enterprise Resource Planning) Systèmes monolithiques couvrant plusieurs modules métier (SAP, Oracle EBS). Ils souffrent d'une rigidité importante et d'un couplage fort entre domaines.

Low-code / No-code Plateformes comme OutSystems ou Mendix permettant de générer des applications à partir de configurations. Elles limitent cependant l'expressivité et le contrôle architectural.

Backend-as-a-Service (BaaS) Services cloud (Firebase, Supabase, Appwrite) offrant des fonctionnalités communes (authentification, base de données, stockage). Ils ne modélisent pas la sémantique métier.

BCaaS Notre approche se positionne entre le BaaS technique et l'ERP fonctionnel : un *Business Core* générique, configurable par domaine, exposé comme un service API.

2.2 Architectures multi-tenant

Le multi-tenancy désigne la capacité d'un système à servir plusieurs clients (tenants) de façon isolée sur une infrastructure partagée. Trois stratégies principales existent (??).

BCAAS adopte, dans son implémentation actuelle, la stratégie par colonne `tenant_id`, qui maximise le partage des ressources tout en garantissant une isolation logique stricte

Table 2.1: Stratégies de multi-tenancy

Stratégie	Isolation	Coût	Complexité
Base de données séparée	Très forte	Élevé	Faible
Schéma séparé (PostgreSQL)	Forte	Moyen	Moyenne
Colonne <code>tenant_id</code>	Logique	Faible	Modérée

(??). Une évolution vers l'isolation par schéma est prévue pour les tenants à forte exigence.

2.3 L'ingénierie des protocoles comme méthode

Le modèle OSI définit sept couches, chacune offrant des services à la couche supérieure et consommant ceux de la couche inférieure. Cette décomposition présente quatre propriétés fondamentales exploitées dans notre conception :

1. **Séparation des responsabilités** : chaque couche a une fonction clairement délimitée.
2. **Encapsulation** : les données traversent les couches en s'enrichissant de métadonnées sans que le contenu applicatif soit altéré.
3. **Interopérabilité** : les interfaces entre couches sont standardisées (protocoles).
4. **Substitution** : une couche peut être remplacée sans impacter les autres.

L'originalité de BCAAS n'est pas de réutiliser ces propriétés comme simple métaphore pédagogique, mais d'en faire une *méthode de conception* : définir les contrats avant le code, modéliser les échanges comme des messages encapsulés, et isoler les préoccupations exactement comme une pile protocolaire.

Chapitre 3

Modèle mathématique et analogie formelle

3.1 Formalisation de la pile protocolaire métier

Nous proposons une formalisation de l'analogie réseau-métier à travers une structure algébrique en couches.

Définition 3.1 (Pile protocolaire métier). Soit $\mathcal{P} = (L_1, L_2, L_3, L_4, L_5)$ une pile protocolaire métier ordonnée telle que L_i offre des services à L_{i+1} et consomme les services de L_{i-1} , avec :

$L_1 =$ Infrastructure	(analogie : couche physique / liaison)
$L_2 =$ Transport & Messaging	(analogie : couche transport)
$L_3 =$ Tenant & Routing	(analogie : couche réseau)
$L_4 =$ Context & Policy	(analogie : couche session / présentation)
$L_5 =$ Business Capabilities	(analogie : couche application)

Définition 3.2 (Paquet métier). Un paquet métier $\mathcal{M}$ est un triplet :

$$\mathcal{M} = (H_T, H_C, \mathcal{P}_{biz})$$

où :

- $H_T = \{h_i \mid h_i \in \mathcal{A}_T\}$ est l'en-tête de transport (version d'API, méthode HTTP, horodatage) ;
- $H_C = \{(k, v) \mid k \in \mathcal{K}_C, v \in \mathcal{V}\}$ est l'en-tête de contexte métier (tenant, acteur,

traçabilité, SLA) ;

- $\mathcal{P}_{biz}$ est le payload métier : données propres à l'opération.

Définition 3.3 (Tenant). Un tenant T_i est un quadruplet :

$$T_i = (\text{id}_i, \mathcal{B}_i, \mathcal{A}_i, \mathcal{R}_i)$$

où id_i est un identifiant UUID unique, $\mathcal{B}_i$ l'ensemble des *businesses* hébergés, $\mathcal{A}_i$ le catalogue d'acteurs et d'utilisateurs, et $\mathcal{R}_i$ l'ensemble des règles métier configurées.

3.2 Table de correspondance réseau-métier

Table 3.1: Correspondance formelle entre concepts réseaux et concepts métier

Concept réseau	Définition réseau	Équivalent métier dans BCaaS
Adresse IP	Identifiant unique d'un nœud	<code>tenant_id</code> + domaine + service
En-tête de paquet	Métadonnées d'acheminement	<code>tenant_id</code> , <code>actor_id</code> , <code>trace_id</code> , SLA
Payload / SDU	Données applicatives	Corps de la requête métier
Encapsulation	Ajout d'en-têtes à chaque couche	Enrichissement du contexte au passage des couches
Routage	Sélection du chemin vers la destination	Résolution <code>tenant</code> → service → instance
NAT / Proxy	Traduction d'adresses	API Gateway : réécriture et normalisation
VLAN	Isolation logique sur réseau partagé	Filtre <code>tenant_id</code> par requête
TCP (fiable)	Transport avec accusé de réception	REST synchrone avec retry et idempotence
UDP (rapide)	Transport sans garantie	Événements asynchrones (Kafka)
Session TCP	État d'une connexion	Contexte de requête, corrélation <code>trace_id</code>
QoS	Priorité et garanties de débit	SLA par tenant, <i>rate limiting</i>
TTL (Time To Live)	Durée de vie d'un paquet	Expiration du token JWT
Control plane	Gestion des routes (BGP, OSPF)	Admin : configuration tenants, règles
Data plane	Acheminement des paquets	Exécution des transactions métier
Firewall	Filtrage de paquets	Spring Security, contrôle d'accès API
Token bucket	Lissage de trafic (<i>policing</i>)	<i>Rate limiting</i> bucket4j

3.3 Modélisation de la requête comme datagramme

Considérons une requête de création d'instance métier. Dans un réseau classique, un datagramme IP contient des champs d'en-tête (adresse source, destination, TTL, protocole) suivis d'un payload. Dans BCAAS, la requête HTTP entrante suit la même structure (??).

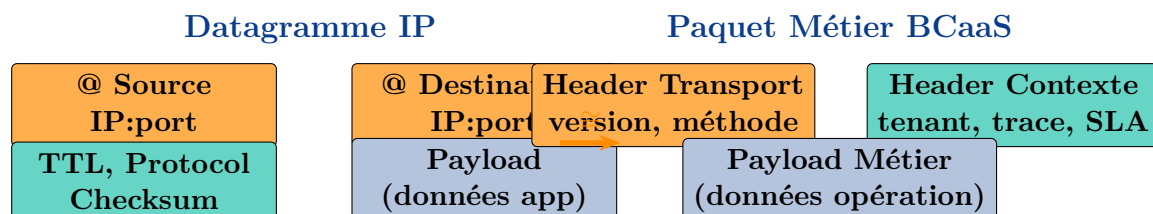


Figure 3.1: Analogie structurelle entre un datagramme IP et un paquet métier BCaaS

3.4 Flux de traitement comme pipeline de protocoles

Le traitement d'une requête dans BCaaS suit le modèle OSI descendant (émetteur) puis ascendant (récepteur), comme l'illustre la ??.

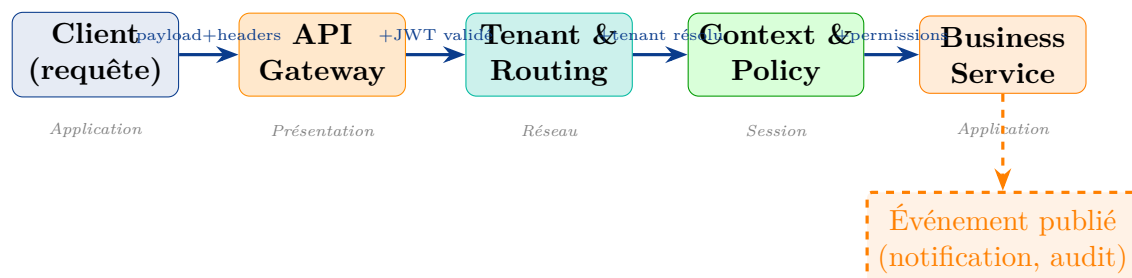


Figure 3.2: Flux bout-en-bout d'une requête dans BCaaS — analogie OSI

3.5 Multi-tenant comme réseau virtuel

Analogie : VLAN ↔ Tenant

Dans un réseau, un VLAN isole logiquement plusieurs groupes de machines sur la même infrastructure physique. Chaque trame VLAN porte un tag (802.1Q) qui identifie son appartenance. Dans BCaaS, chaque requête authentifiée porte un `tenant_id` (issu du token JWT) qui isole les données, les règles et les ressources d'un client sur l'infrastructure partagée.

VLAN tag $\equiv$ `tenant_id`, Switch L2 $\equiv$ `TenantFilter`, Réseau physique $\equiv$ Infrastructure partagée

La propriété d'isolation se formalise ainsi. Soient T_i et T_j deux tenants distincts ($i \neq j$).

On exige :

Propriété 3.1 (Isolation inter-tenant).

$$\forall r \in \mathcal{R}, \text{tenant}(r) = T_i \implies r \notin \mathcal{V}_{T_j}$$

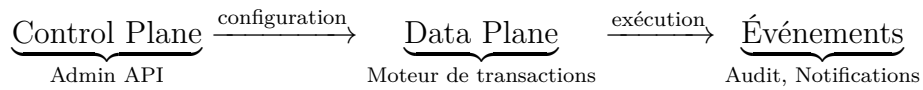
où $\mathcal{V}_{T_j}$ est l'ensemble des ressources visibles par le tenant T_j .

Cette propriété est garantie par la présence systématique de la contrainte `tenant_id = :tid` dans toutes les requêtes, valeur injectée au niveau du `TenantContext` (??).

3.6 Control plane et data plane

Analogie : SDN $\leftrightarrow$ BCaaS Admin

Dans les réseaux définis par logiciel (SDN), le *control plane* configure les règles de routage de façon centralisée, tandis que le *data plane* achemine les paquets. Cette séparation permet de modifier le comportement du réseau sans interruption de service. Dans BCaaS, l'interface d'administration (control plane) configure tenants, règles métier et catalogues ; le moteur d'exécution (data plane) traite les transactions en appliquant ces configurations.



Chapitre 4

Architecture globale de la solution

4.1 Vue d'ensemble

L'architecture de BCAAS est organisée autour de cinq couches protocolaires, directement inspirées du modèle OSI, et d'une séparation claire entre control plane et data plane (??).

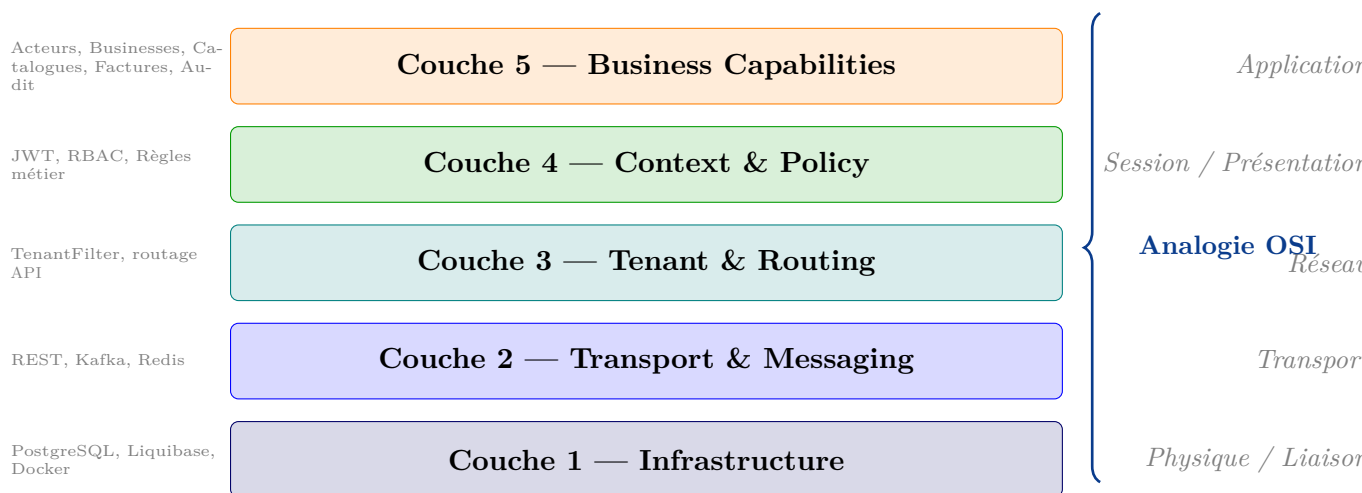


Figure 4.1: Pile protocolaire métier de BCAAS et correspondance avec le modèle OSI

4.2 Description des couches

4.2.1 Couche 1 — Infrastructure

Substrat technique. Elle regroupe la base de données PostgreSQL avec ses migrations Liquibase, le cache Redis et le bus de messages Apache Kafka. Elle correspond aux couches physique et liaison du modèle OSI: elle transporte les données sans connaître leur signification.

4.2.2 Couche 2 — Transport & Messaging

Analogie directe avec la couche transport OSI (TCP/UDP). Elle gère le mode de transport des échanges : synchrone via REST (analogie TCP, fiable et ordonné) et asynchrone via Kafka pour les notifications (analogie UDP, meilleur effort).

4.2.3 Couche 3 — Tenant & Routing

Analogie avec la couche réseau. Le `TenantFilter` joue le rôle d'un routeur : il détermine, à partir du token JWT de la requête, l'identité du tenant et configure le contexte d'exécution pour router implicitement chaque accès aux données vers le bon tenant.

4.2.4 Couche 4 — Context & Policy

Analogie avec les couches session et présentation. Cette couche valide le token JWT, décode les rôles et permissions, et applique les politiques de sécurité (RBAC). L'en-tête métier (H_C) y est construit.

4.2.5 Couche 5 — Business Capabilities

Couche applicative. Elle contient les services métier : gestion des acteurs, des businesses, des ressources, des catalogues de services, des demandes de service, des factures, des documents et de l'audit.

4.3 Architecture : monolithe modulaire

BCAAS n'est pas déployé comme un maillage de microservices, mais comme un **monolithe modulaire** : les modules métier sont organisés en packages cohérents et communiquent par appels directs en mémoire. Ce choix conserve la séparation logique des responsabilités et l'architecture hexagonale tout en évitant la complexité opérationnelle d'un mesh distribué — adapté à un prototype académique tout en restant évolutif (??).

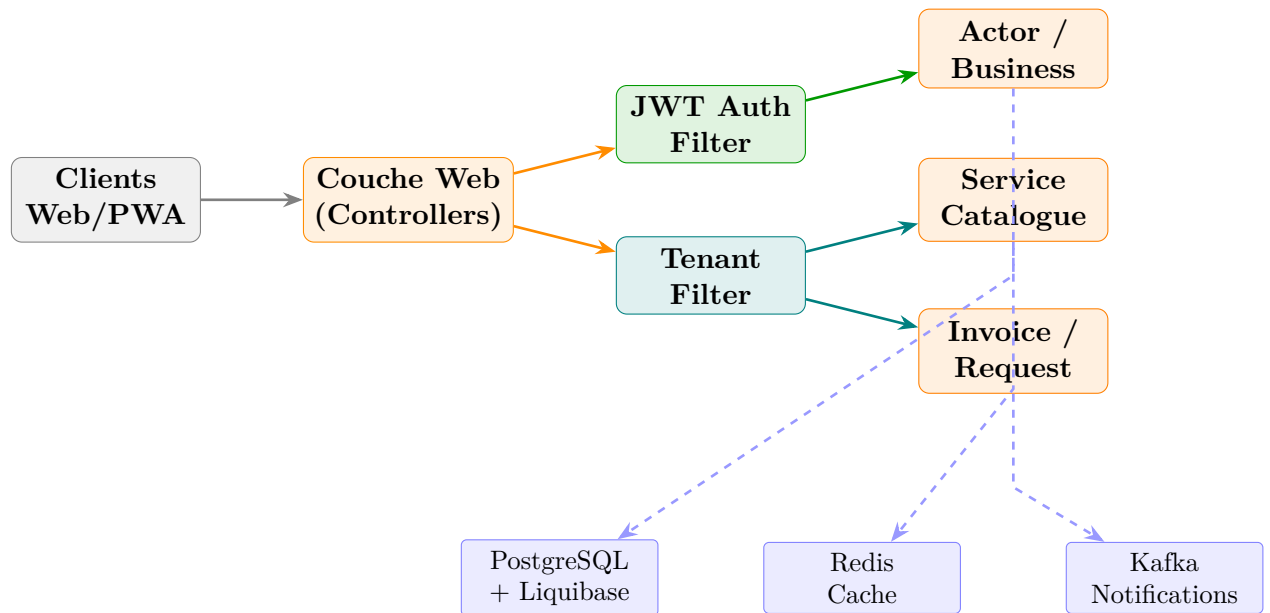


Figure 4.2: Architecture modulaire de référence de BCAAS

Chapitre 5

Conception détaillée

5.1 Modèle de données

Le cœur de BCAAS est un modèle de données concret couvrant les besoins transverses de toute activité de service : acteurs, businesses, catalogues, demandes, facturation, contenus et traçabilité. Le modèle s'articule autour de quinze entités (??).

Table 5.1: Entités du modèle de données — Business Core

Entité	Attributs principaux
tenant	id, name, isActive, createdAt
user	id, firstName, lastName, email, gender, password, isActive, country, phone, #tenant
actor	id, nomProfessionnel, speciality, bio, role, createdAt, #user
business	id, nameBusiness, domain, typeOfInvolvedActors, requiredJobProfile, createdAt
business_rule	id, ruleKey, ruleValue, description, createdAt
resource	id, nameRes, type, quantity, description, disponibility, createdAt
service_catalogue	id, nameSerCatalogue, description, currency, isAvailable, createdAt
service_request	id, traceId, serviceName, description, status, #actor, #portfolio
portfolio	id, title, description, status, #business
invoice	id, amount, currency, status, usedAt, paidAt, issueAt, #serviceRequest
media	id, type, url, label, createdAt, #business
message	id, content, isRead, createdAt, #serviceRequest
notification	id, title, message, isRead, createdAt, #serviceRequest
audit_event	id, action, entity, actor, timestamp, tenant
supported_currency	code, label, symbole

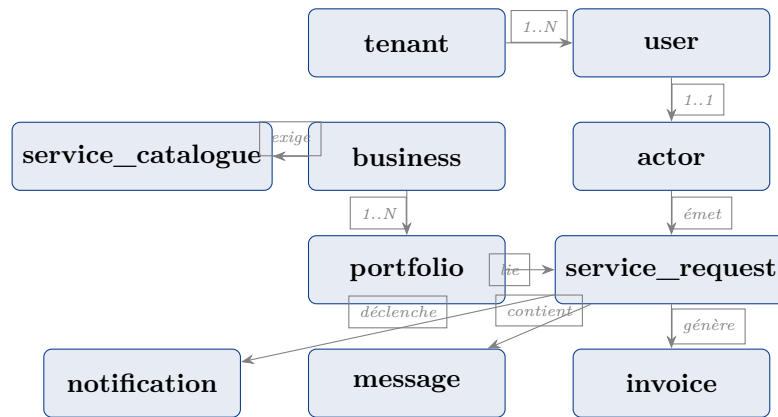


Figure 5.1: Diagramme entité-relation simplifié du modèle métier

5.1.1 Diagramme entité-relation simplifié

5.2 Conception de l'API REST

L'API suit les principes REST et est organisée par ressource métier. Les seize contrôleurs exposent les points d'entrée listés au ?? (extrait représentatif).

Table 5.2: Endpoints principaux de l'API BCAAS

Méthode	Endpoint	Description	Couche
POST	/api/auth/register	Inscription d'un utilisateur	L4
POST	/api/auth/login	Émission d'un token JWT	L4
POST	/api/tenants/register	Provisionnement d'un tenant	L3
GET	/api/tenants	Liste des tenants	L3
POST	/api/businesses	Création d'un business	L5
POST	/api/service-catalogues	Ajout au catalogue de services	L5
POST	/api/service-requests	Création d'une demande de service	L5
GET	/api/service-requests/{id}/messages	Fil de messages d'une demande	L5
POST	/api/actors	Enregistrement d'un acteur	L5
POST	/api/invoices	Émission d'une facture	L5
POST	/api/business-rules	Définition d'une règle métier	L4
GET	/api/audit	Consultation du journal d'audit	L5
GET	/api/analytics	Statistiques (ré-servé admin)	Adm.

Chapitre 6

Protocole d'échange pour un cas métier concret

6.1 Cas choisi : création d'un business et ajout au catalogue

Ce cas métier illustre la séquence complète d'enregistrement d'une nouvelle activité professionnelle (ex. : auto-école) dans un tenant, puis l'ajout de services à son catalogue. Le processus correspond à deux requêtes distinctes mais liées.

6.1.1 Protocole : création d'un business

POST /api/businesses

En-têtes obligatoires :

- Authorization: Bearer <JWT> — authentification (le claim `tenantId` porte l'identité du tenant, analogie : adresse réseau + ticket de session) ;
- Content-Type: application/json.

```
1 {  
2   "name": "Auto-Ecole Centrale",  
3   "domain": "driving_school",  
4   "description": "Numero 1 dans la formation auto",  
5   "typeOfInvolvedActors": ["instructor", "student"],  
6   "requiredJobProfile": "moniteur"  
7 }
```

Listing 6.1: Corps de la requête (payload)

```
1 {
```

```
2  "id": "550e8400-e29b-41d4-a716-446655440000",
3  "nameBusiness": "Auto-Ecole Centrale",
4  "status": "ACTIVE",
5  "createdAt": "2025-05-04T10:00:00Z"
6 }
```

Listing 6.2: Réponse (succès — 201 Created)

Codes d'erreur (analogie ICMP) :

- 400 Bad Request : payload invalide ;
- 401 Unauthorized : JWT manquant ou invalide ;
- 403 Forbidden : rôle insuffisant ;
- 409 Conflict : business déjà existant.

6.1.2 Protocole : ajout au catalogue de services

```
1 {
2   "businessId": "550e8400-e29b-41d4-a716-446655440000",
3   "name": "Cours de conduite",
4   "description": "Formation theorique et pratique de 3 mois",
5   "basePrice": 150000,
6   "currency": "XAF",
7   "isAvailable": true
8 }
```

Listing 6.3: POST /api/service-catalogues — corps

Sémantique du transport :

- Mode : **synchrone** (REST) — analogie TCP ;
- Timeout : 30 secondes ;
- Idempotence : non (chaque POST crée une nouvelle ressource) ;
- Retry : recommandé avec backoff exponentiel (max 3 tentatives).

Chapitre 7

Stratégie multi-tenant et isolation

7.1 Stratégie actuelle : colonne tenant_id

Dans l'implémentation actuelle, tous les tenants partagent les mêmes tables PostgreSQL. L'isolation est assurée par la présence systématique d'un `tenant_id` de type UUID, dont la valeur est extraite du token JWT et injectée dans un `ThreadLocal` pour la durée exacte de la requête.

```
1 @Component
2 @Order(2) // s'exécute APRES JwtAuthFilter (Order 1)
3 public class TenantFilter extends OncePerRequestFilter {
4
5     private final JwtService jwtService;
6
7     @Override
8     protected void doFilterInternal(HttpServletRequest req,
9                                     HttpServletResponse res, FilterChain chain)
10        throws ServletException, IOException {
11         try {
12             String authHeader = req.getHeader("Authorization");
13             if (authHeader != null && authHeader.startsWith("Bearer
14
15
16
17
18
19
20
21
22
23
14 String jwt = authHeader.substring(7);
15 String tenantId = jwtService.extractTenantId(jwt);
16 if (tenantId != null && !tenantId.isBlank()) {
17     // Analogie : lecture du tag VLAN sur le paquet
18     TenantContext.setTenantId(UUID.fromString(tenantId));
19 }
20 }
21 chain.doFilter(req, res);
22 } finally {
23     // CRITIQUE : liberation du ThreadLocal (analogie : fin
```

```

24         de session)
25         TenantContext.clear();
26     }
27 }

```

Listing 7.1: TenantFilter — résolution du tenant depuis le JWT (Couche 3)

Garantie d'isolation

Le `TenantContext` expose le `tenant_id` courant à tous les services via un `ThreadLocal`. Chaque thread du pool dispose de sa propre copie, et le bloc `finally` garantit son nettoyage en fin de requête — éliminant toute fuite inter-tenant, même en cas d'exception.

7.2 Comparaison et perspective

Table 7.1: Comparaison des stratégies d'isolation

Stratégie	Isolation	Performance	Complexité
Colonne <code>tenant_id</code>	Logique	Bonne (index)	Faible
Schéma séparé	Physique	Très bonne	Moyenne
Base de données séparée	Totale	Variable	Élevée

Pour les tenants à exigences renforcées (données sensibles, conformité, volumétrie élevée), une isolation par schéma PostgreSQL dédié est envisagée. La résolution se ferait dynamiquement via un `SET search_path TO tenant_<id>, public;` exécuté en début de requête — analogue au routage vers un VLAN dédié.

7.3 Provisionnement d'un nouveau tenant

Le provisionnement suit une séquence protocolaire, analogue à l'attribution d'une adresse et à la configuration d'un VLAN :

1. Réception de la requête `POST /api/tenants/register` ;
2. Validation des informations (nom) ;
3. Génération d'un `tenant_id` unique (UUID v4) ;
4. Création de l'enregistrement `tenant` et de l'utilisateur administrateur ;
5. Émission d'un token JWT portant le claim `tenantId` ;
6. Retour de la configuration au client.

Chapitre 8

Sécurité

8.1 Filtrage en couches

La sécurité de BCAAS s'inspire directement du concept de firewall réseau, avec un filtrage à plusieurs niveaux :

1. **Niveau transport (L2)** : HTTPS en production (TLS).
2. **Niveau authentification (L4)** : le `JwtAuthFilter` (Order 1) valide le token JWT (jjwt 0.12.6) et établit l'authentification Spring Security.
3. **Niveau routage (L3)** : le `TenantFilter` (Order 2) résout le tenant et borne la visibilité des données — analogie avec le filtrage par appartenance VLAN.
4. **Niveau autorisation (L4)** : contrôle d'accès basé sur les rôles (`USER`, `ADMIN`, `PROVIDER`, `CONSUMER`) ; les routes `/api/admin/**` requièrent le rôle `ADMIN`, les endpoints d'authentification et d'enregistrement sont publics.
5. **Niveau applicatif (L5)** : validation métier (Bean Validation) sur les payloads.

8.2 TTL du token comme analogie réseau

La durée de vie du token JWT (TTL_{JWT}) est formellement analogue au champ TTL d'un paquet IP :

$$TTL_{JWT} = t_{\text{émission}} + \Delta t_{\text{validité}}$$

Au-delà de cet horizon, le token est rejeté par le `JwtAuthFilter`, exactement comme un paquet dont le TTL atteint zéro est détruit par un routeur.

Chapitre 9

Observabilité et QoS métier

9.1 Les trois piliers

L'observabilité dans BCAAS suit les trois piliers classiques — logs, métriques, traces — chacun mis en correspondance avec un concept réseau.

9.1.1 Logs structurés

```
1 {  
2   "timestamp": "2025-05-04T10:05:23Z",  
3   "level": "INFO",  
4   "tenant_id": "550e8400-e29b-41d4-a716-446655440000",  
5   "trace_id": "7e9a8b7c-6d5e-4f3a-2b1c-9d8e7f6a5b4c",  
6   "service": "BusinessService",  
7   "operation": "createBusiness",  
8   "duration_ms": 145,  
9   "http_status": 201  
10 }
```

Listing 9.1: Exemple de log structuré (JSON)

9.1.2 Métriques

Spring Boot Actuator expose l'état de santé et les métriques de l'application, et le service `AnalyticsService` calcule des indicateurs métier (??). Leur exposition vers Prometheus/Grafana constitue une perspective directe.

9.1.3 Tracing distribué

L'identifiant `trace_id` (champ `traceId` de `service_request`) joue le rôle du numéro de séquence TCP : il permet de corréler les étapes d'une transaction de bout en bout.

Table 9.1: Métriques métier et analogies réseau

Métrique	Analogie réseau	Calcul
Taux de conversion	Taux de livraison de paquets	requêtes honorées / total
Latence moyenne	RTT (Round Trip Time)	temps moyen de traitement
Débit (req/s)	Bande passante	requêtes par minute
Taux d'erreur	Taux de perte	4xx/5xx / total
Panier moyen	Taille moyenne de payload	chiffre d'affaires / nb factures

9.2 QoS métier : niveaux différenciés

Inspirée des classes de service DiffServ des réseaux IP, BCAAS prévoit des niveaux de QoS par tenant, appuyés sur un mécanisme de *rate limiting* (token bucket via la dépendance `bucket4j-redis`) — l'analogue applicatif du *traffic policing*.

Table 9.2: Niveaux de QoS proposés

Niveau	Débit garanti	Priorité	Isolation
BRONZE	10 req/s	Standard	Colonne <code>tenant_id</code>
SILVER	50 req/s	Prioritaire	Colonne + cache Redis
GOLD	200 req/s	Dédiée	Schéma dédié (cible)

Chapitre 10

Implémentation technique

10.1 Pile technologique

Table 10.1: Pile technologique de BCAAS

Composant	Technologie	Version
Framework backend	Spring Boot	3.5.12
Langage	Java	21 LTS
Base de données	PostgreSQL	16+
Migrations	Liquibase	7 changelogs
Sécurité	Spring Security + JWT (jjwt)	0.12.6
Cache	Redis (Spring Data Redis)	—
Messagerie	Apache Kafka (Spring Kafka)	—
Rate limiting	bucket4j-redis	8.10.1
Documentation API	SpringDoc OpenAPI	2.8.6
Observabilité	Spring Boot Actuator	—
Build	Maven	3.9+
Tests	JUnit 5 + Mockito	18 fichiers
Conteneurisation	Docker + Docker Compose	—
Frontend	Next.js / React	16.2 / 19

10.2 Structure du projet backend

Le projet suit l'architecture hexagonale (*ports et adaptateurs*) combinée au DDD. L'arborescence réelle est la suivante :

```
1 backend/
2     pom.xml
3     Dockerfile
4     docker-compose.yml
5     src/main/java/com/bizcore/bizcore_backend/
```

```

6         auth/           # AuthController (register, login)
7         config/         # Jwt, Kafka, Redis, Security, OpenApi
8         controller/     # 16 controllers REST (Couche 3)
9         domain/         # 15 entites JPA (Couche 5)
10        dto/            # Objets de transfert
11        exception/      # Gestion centralisee des erreurs
12        listener/       # NotificationKafkaListener
13        repository/     # Repositories Spring Data
14        security/       # JwtAuthFilter, TenantFilter,
TenantContext
15        service/        # 14 services metier (Couche 4-5)
16        src/main/resources/
17            application-dev.yml / application-prod.yml
18            db/changelog/ # 7 migrations Liquibase
19        src/test/       # 18 fichiers de tests (JUnit 5 +
Mockito)

```

Listing 10.1: Structure du projet backend BCaaS

10.3 Le paquet métier comme abstraction

La structure conceptuelle du « paquet métier » se reflète dans la composition d’une requête authentifiée : en-tête de transport (méthode, version), en-tête de contexte (tenant, trace, rôles) et payload métier.

```

1 public record BusinessPacket<T>(  

2     TransportHeader transportHeader, // analogie : en-tete IP  

3     ContextHeader contextHeader,    // analogie : en-tete applicatif  

4     T payload                        // analogie : SDU  

5 ) {  

6     public record TransportHeader(String version, String method,  

7         Instant timestamp) {}  

7     public record ContextHeader(String tenantId, String actorId,  

8         String traceId, String roleScope) {}  

9 }

```

Listing 10.2: BusinessPacket — représentation conceptuelle du paquet métier

Chapitre 11

Frontend Next.js

11.1 Architecture du frontend

Le frontend de BCAAS est développé avec Next.js 16 (App Router) et React 19. Il matérialise le plan de données vu de l'extérieur :

Site vitrine (data plane view) présentation publique de l'API, documentation interactive et assistant conversationnel.

Tableau de bord Admin (control plane) interface sécurisée pour la gestion des tenants, businesses, acteurs, services, demandes et factures. *Prévu dans une version future, seules les ébauches de pages sont présentes dans le code actuel.*

11.2 Choix techniques

Table 11.1: Technologies frontend

Besoin	Technologie	Justification
Framework	Next.js 16 (App Router)	SSR/SSG, SEO, PWA
UI / React	React 19	Dernières primitives concurrentes
Styling	Tailwind CSS	Rapidité, cohérence
Composants	shadcn/ui (Radix)	Accessibilité, personnalisable
Animations	Framer Motion	Transitions fluides
3D / visuels	three.js + react-three-fiber	Scènes interactives
État global	Zustand	Simple, performant
Données serveur	TanStack Query + Axios	Cache, synchronisation API
Coloration code	Shiki	Documentation API

11.3 Pages principales

11.3.1 Site vitrine et documentation

- / — page d'accueil présentant l'approche protocolaire ;
- /docs et /docs/[id] — documentation de l'API ;
- /guides et /guides/[slug] — guides d'utilisation ;
- /chat — assistant conversationnel (IA) appuyé sur /api/chat ;
- /demo — démonstration interactive.

11.3.2 Tableau de bord admin (à venir)

Les ébauches de pages listées ci-dessous sont présentes dans le code, mais l'intégration fonctionnelle complète du tableau de bord est repoussée à une version ultérieure :

- /dashboard — vue globale ;
- /dashboard/businesses, /dashboard/actors, /dashboard/services ;
- /dashboard/requests, /dashboard/invoices ;
- /dashboard/settings — configuration.

11.4 Déploiement

Le frontend est déployé sur **Vercel** et le backend sur **Render** (conteneur Docker, base PostgreSQL managée). La configuration de déploiement est centralisée (`render.yaml`), et l'URL du backend est paramétrée via une variable d'environnement unique côté frontend.

Chapitre 12

Tests et validation

12.1 Stratégie de tests

La stratégie de tests suit la logique en couches et compte **18 fichiers de tests** (JUnit 5 + Mockito) :

Tests unitaires (L4-L5) services métier : `BusinessServiceTest`, `InvoiceServiceTest`, `ActorServiceTest`, `ServiceCatalogueServiceTest`, `ServiceRequestServiceTest`, `BusinessRuleServiceTest`, `TenantServiceTest`, `PersonServiceTest`.

Tests de couche web contrôleurs avec `@WebMvcTest` : `BusinessControllerTest`, `InvoiceControllerTest`, `ActorControllerTest`, `ServiceRequestControllerTest`.

Tests d'intégration (L2-L3) `ServiceRequestIntegrationTest`, `SecurityIntegrationTest`.

Tests de sécurité et de contexte `JwtServiceTest`, `TenantContextTest` (vérification de l'isolation `ThreadLocal`).

12.2 Scénario de test d'intégration

```
1 @SpringBootTest(webEnvironment = RANDOM_PORT)
2 @AutoConfigureMockMvc
3 class BusinessIntegrationTest {
4
5     @Test
6     void shouldCreateBusinessForAuthenticatedTenant() throws
7         Exception {
8         var request = ""
9         { "name": "Auto-Ecole Centrale", "domain": "driving_school" }
10         "" ;
11         // analogie : envoi d'un paquet reseau
```

```
11      mockMvc.perform(post("/api/businesses")
12                      .header("Authorization", "Bearer " + jwt)
13                      .contentType(APPLICATION_JSON)
14                      .content(request))
15      // analogie : accuse de reception (ACK)
16      .andExpect(status().isCreated())
17      .andExpect(jsonPath("$.id").exists())
18      .andExpect(jsonPath("$.status").value("ACTIVE"));
19  }
20 }
```

Listing 12.1: Test d'intégration — création d'un business

Chapitre 13

Conclusion et perspectives

13.1 Bilan

Ce projet a démontré qu’une analogie rigoureuse entre l’ingénierie des protocoles réseaux et la conception d’une plateforme métier n’est pas une simple métaphore pédagogique : elle constitue une méthode de conception puissante. Les concepts de couches, d’encapsulation, de routage, d’adressage, de QoS et de séparation control/data plane ont guidé chaque décision architecturale de BCAAS.

Les réalisations principales sont :

- une pile protocolaire métier en cinq couches formellement liée au modèle OSI ;
- un modèle de données concret de quinze entités couvrant acteurs, businesses, catalogues, demandes, facturation et traçabilité ;
- une API REST documentée (SpringDoc) servie par un monolithe modulaire Spring Boot 3.5 / Java 21 ;
- une authentification JWT multi-tenant avec isolation par `ThreadLocal` et contrôle d’accès par rôles ;
- l’intégration effective de Redis (cache) et Kafka (notifications) ;
- un frontend Next.js 16 / React 19 (site vitrine, documentation, assistant IA) déployé sur Vercel/Render. Le tableau de bord d’administration est en cours de développement.

13.2 Perspectives

Court terme activation du *rate limiting* bucket4j par niveau de QoS et exposition des métriques Actuator vers Prometheus/Grafana.

Moyen terme isolation par schéma PostgreSQL dédié pour les tenants premium ; enrichissement du tracing distribué.

Long terme extraction progressive de microservices à partir des modules existants ; marketplace d'instances métier permettant à des entreprises de déployer leur propre configuration BCAAS sans développement.

13.3 Apports personnels

Ce projet a permis d'acquérir des compétences concrètes en architecture logicielle, en conception d'API REST professionnelle, en sécurité applicative (JWT, RBAC, multi-tenant) et en modélisation de systèmes génériques. La démarche de pensée protocolaire — définir les contrats avant le code, modéliser les échanges comme des messages encapsulés — constitue un réflexe d'ingénieur directement applicable à tout projet de systèmes d'information.

Bibliographie

- [1] A. S. Tanenbaum, D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011.
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- [3] S. Newman, *Building Microservices*, 2nd ed., O'Reilly, 2021.
- [4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [5] C. Richardson, *Microservices Patterns*, Manning, 2018.
- [6] Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot>, consulté en 2026.
- [7] SpringDoc OpenAPI Documentation, <https://springdoc.org>, consulté en 2026.
- [8] Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- [9] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
- [10] ISO/IEC 7498-1, *Information technology — Open Systems Interconnection — Basic Reference Model*, ISO, 1994.